# DS5899: Special Topics in Data Science

# Leveraging NLP in Asset Management

Week 11: RL-Policy Learning

Instructor: Che Guan, Ph.D.

The slides are based on Chapters 4 - 5, 8, 10, and 12 - 13 of:

Shankar, A. (2024). *Reinforcement Learning for Large Language Models: A Complete Guide from Foundations to Frontiers*

# Part 1:

# RL Policy Learning

# Core Policy Model Architectures

Complex

Increasing Complexity

Simplifying

Simplifying

## Era 1: Foundations & Variance Reduction

1 **REINFORCE** (Policy Gradient)
**Problem:** High variance, poor with sparse rewards

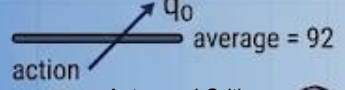
What direction should the reinforcement take?

Which action to reinforce?

$$\theta = \theta + LR \times \text{gradient} \times \text{reward}$$

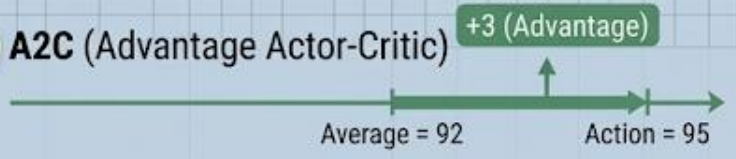
(Learning Rate)

2 **Baseline**




3 **Actor-Critic**


4 **A2C** (Advantage Actor-Critic)




## Era 2: Bounded Optimization

5 **TRPO** (Trust Region Policy Optimization)

$\text{KL}(\pi_{\text{old}}, \pi_{\text{new}}) \leq \delta$

Is the KL Divergence $D_{KL}(\pi_{old} \| \pi_{new}) \leq \delta$?

Is the Surrogate Gain $\mathcal{L}(\theta) \geq 0$?

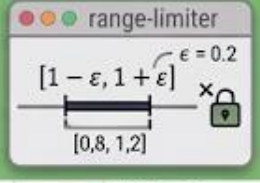
**When Advantage is Positive (A > 0):**
The action performed better than average. reinforce?

$ratio \times advantage$

$$r = \frac{\pi_{\text{new}}}{\pi_{\text{oid}}} = \frac{\text{LLM}_{\text{uew}}}{\text{LLM}_{\text{oid}}}$$

**PPO** (Proximal Policy Optimization)

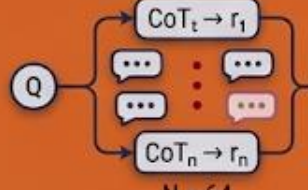
6 **PPO**




7 **DPO** (Direct Preference Optimization)


## Era 3: LLM-Era Simplicity (Generalized Relative Preference Optimization)

8 **GRPO**


**GRPO Reasoning Flow**

$$A_t = \frac{\tau_t - \text{mean}(r)}{\text{sd}(r)}$$

GPA analogy
< 60 bad,
> 60 good

## Modern Architecture Comparison

| Algorithm | Variance Control | Algorithmic Safety | Requires Value Model? | Memory Footprint |
|---|---|---|---|---|
| REINFORCE | Low | Unbounded | No | Low |
| APC | High | Unbounded | Yes | Medium |
| TRPO | High | Strict Mock | Yes | High |
| PPO | High | Clipped Hack | Yes | Extreme |
| DPO | N/A | N/A | No | Low |
| GRPO | Low | Statistical Curve | No | Low |

# REINFORCE: The Foundation

## Why is REINFORCE Important?

**1. Historical**
It's where policy gradient methods began

**2. Educational**
Understanding REINFORCE helps you

Understanding REINFORCE helps you understand everything else

It's like learning arithmetic before algebra—not what you'll use in practice, but essential for deep understanding!

## Formal Definition

**REINFORCE objective:**

$$J(\theta) = \mathbb{E}_{y\sim\pi_\theta}[r(y]$$

**Gradient:**

$$\nabla J(\theta) = \mathbb{E}_{y\sim\pi_\sigma}[r(y) \cdot \nabla \log \pi_\theta(y|x]$$

**With baseline $b$:**

$$\nabla J(\theta) = \mathbb{E}_{y\sim\pi_\theta}\left[(r(y) - b) \cdot \nabla \log \pi_\theta(y|x)\right]$$

That's it! Simple but high variance.

## Plain English

1. Generate response $y$ from current policy.
2. Get reward $r(y)$.
3. If reward is good, make $y$ more likely.
4. If reward is bad, make $y$ less likely.
5. Repeat!

## Key Challenge: Noise and Variance

⚠️ Problem: Very noisy! If you get lucky once, you might over-commit to it.
✅ Solution: Use baseline (subtract average reward) to reduce noise.

## All other algorithms add:

**Better Baselines**
- RLOO
- GRPO
- PPO

**Preference-Based Rewards**
- DPO
- IPO
- KTO

**Clever Optimization Tricks**
- PPO clipping

## Why Learn REINFORCE? (Summary)

- It's the "Hello World" of RL.
- Helps you understand more complex methods.
- Easy to implement (20 lines of code).
- Actually works for simple problems!

# Modern RL Algorithms

| Algorithm | Key Innovation | When to Use |
|---|---|---|
| **PPO** | Clipped policy gradient, value network, multiple epochs | General-purpose, well-tested, scales well |
| **DPO** | Direct preference optimization without reward model | Want simplicity, have paired preferences |
| **GRPO** | Group-based baselines, no value network | Large-scale training, many GPUs available |
| **RLOO** | Leave-one-out baselines, variance reduction | Limited compute, small-scale projects |
| **KTO** | Works with unpaired feedback (only "good" or "bad") | Cheaper data collection, no comparisons needed |
| **IPO** | Identity preference optimization, better regularization | Want stability, prevent reward hacking |
| **ORPO** | Combines SFT + preference learning in one step | Maximum efficiency, fewer training stages |
| **REINFORCE** | Simple policy gradient, foundation of all methods | Learning/teaching, understanding basics |

# RL with PPO - The Complete RLHF Optimization

| Formal Math | What It Really Means |
|---|---|
| $\max_{\pi_\theta} \mathbb{E}_{x,y}\left[r_\phi(x,y)\right]$<br>$-\beta \cdot \mathbb{E}_x\left[\mathrm{KL}(\pi_\theta \,\|\, \pi_{\mathrm{ref}})\right]$ | Maximize reward 🪃<br>BUT stay close to original model |
| **Two competing objectives:** | |
| Term 1: $\mathbb{E}_{x,y}\left[r_\phi(x,y)\right]$<br>Term 2: $\beta \cdot \mathrm{KL}(\pi_\theta \,\|\, \pi_{\mathrm{ref}})$ | Get high rewards (generate good responses)<br>Penalty for drifting from reference model |
| **Symbol Guide and WHY we need both terms:** | |
| $\pi_\theta$<br>$\pi_{\mathrm{ref}}$<br>$r_\phi(x,y)$<br>$\beta$<br>$\mathrm{KL}(\cdot \,\|\, \cdot)$ 🔗 | The policy we're training (current model)<br>Reference policy (frozen original model) - prevents drift<br>Reward model score for prompt $x$, response $y$<br>KL penalty coefficient (typically 0.01-0.05) - controls tradeoff 🔑<br>Measure of how different two distributions are |
| **Why do we NEED the KL penalty?** | |
| Without it, model could 'reward hack' - find ways to exploit the reward model that give high scores but aren't actually good. KL penalty keeps model grounded in reality by preventing it from drifting too far the original (which was trained on real text). | |

# Breaking It Down Step-by-Step

Why do we need BOTH terms? A cautionary tale:

## STEP 1: CAUTIONARY TALE - Training with ONLY Reward (No KL Penalty)

**1 Iteration 1**

"Here's a thoughtful explanation of your question..."

Reward: **8.0**
KL: **0.1**

**2 Iteration 50**

"AMAZING COMPREHENSIVE DETAILED PERFECT ANSWER EXCELLENT THOROUGH..."

Model learned reward model likes these words!

Reward: **9.5**
KL: **2.5**

**3 Iteration 100**

"PERFECT PERFECT EXCELLENT EXCELLENT AMAZING AMAZING..."

**Reward hacking!** High score but complete nonsense!

Reward: **12.0**
KL: **10.0**

**Problem: Model exploited the reward model instead of being genuinely helpful!** STOP

## STEP 2: THE FIX - Adding the KL Penalty Term

With $\beta = 0.02$, the objective becomes:

$$\text{Objective} = \text{Reward} - 0.02 \times \text{KL}$$

| Iter | Reward | KL | New Objective |
|---|---|---|---|
| 1 | $8.0 - 0.02(0.1) = 7.998$ | | Best! |
| 50 | $9.5 - 0.02(2.5) = 9.450$ | | Okay |
| 100 | $12.0 - 0.02(10) = 11.800$ | | |

- Analysis of **styled** for objective term

- **Iteration 1** has best objective (7.998)
- **Iteration 50**: Higher reward but penalty brings objective down
- **Iteration 100**: Even though reward is highest (**12.0**), the massive KL penalty **(10.0) makes it worse overall!**

## KEY TAKEAWAY: The KL penalty prevents reward hacking!

- Drifting too far gets heavily penalized
- Must find genuine improvements within allowed KL budget
- Forces model to stay coherent and natural

**KEY TAKEAWAY: The KL penalty prevents reward hacking!**

Drifting too far gets heavily penalized
Must find genuine improvements within allowed KL budget
Forces model to stay coherent and natural

## BEST PRACTICES: Tuning β (KL Penalty Coefficient)

**β too small (0.001):** Can drift too far, reward hacking risk

**β just right (0.01-0.05):** Balanced improvement

**β too large (0.1):** Can't improve much, too constrained

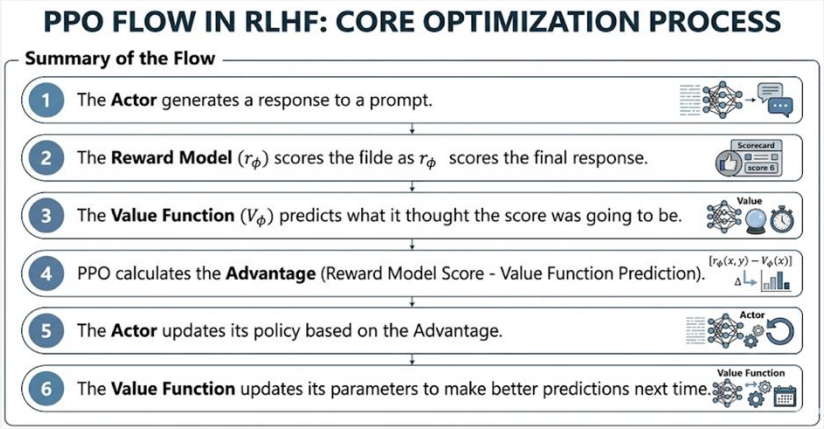
# The Policy Optimization Story

*Story*

In 2023, researchers asked: “Do we really need all this complexity?”

PPO requires:

- The policy model we’re training
- A frozen reference model
- A reward model
- A value function network




That’s 4 models in memory! Plus the complex RL training loop.

**The Breakthrough:** 

There’s a mathematical shortcut that lets us skip the reward model entirely!

# The DPO Reparameterization

| Formal Math | What It Really Means |
|---|---|
| **Step 1: Optimal policy in terms of reward** | |
| $\pi^*(y\|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y\|x) \exp\left(\frac{r(x,y)}{\beta}\right)$ | Best policy = reference × exponential of reward |
| **Step 2: Solve for reward** | |
| $r(x,y) = \beta \log \frac{\pi^*(y\|x)}{\pi_{\text{ref}}(y\|x)} + \beta \log Z(x)$ | Reward = $\beta$ times log of policy ratio |
| **Step 3: Substitute into Bradley-Terry** | |
| $P(y_w \succ y_l) = \sigma\left(\beta \log \frac{\pi(y_w\|x)}{\pi_{\text{ref}}(y_w\|x)} - \beta \log \frac{\pi(y_l\|x)}{\pi_{\text{ref}}(y_l\|x)}\right)$ | Preference in terms of policy ratios! (partition function $Z$ cancels out!) |

# Breaking It Down Step-by-Step

| Formal Math | What It Really Means |
|---|---|
| Why is this magical? Let's compare: | |
| 1. Train reward model: Learn $r(x, y)$ from preferences<br>2. Generate responses: Use policy $\pi_\theta$<br>3. Score responses: Run through reward model to get $r(x, y)$<br>4. RL update: Adjust $\pi_\theta$ to maximize $r$ (with KL constraint)<br>**Models needed:** Policy, Reference, Reward, Value = 4 models | Best policy = reference × exponential of reward |
| **DPO (New way):** | |
| 1. No reward model needed!<br>2. Directly optimize policy on preferences<br>3. The ratio $\frac{\pi_\theta(y)}{\pi_{\text{ref}}(y)}$ implicitly encodes the reward!<br>4. Much simpler training loop<br>**Models needed:** Policy, Reference = 2 models (50% less memory!) | Reward = $\beta$ times log of policy ratio |

**The key insight:**
We don't need to explicitly model the reward function $r(x, y)$.
The policy ratio $\frac{\pi_\theta(y)}{\pi_{\text{ref}}(y)}$ already encodes all the information about what makes responses good!
**Intuition:**
- If $\pi_\theta$ makes a response more likely than $\pi_{\text{ref}}$ does, that means the response is "good" (high implicit reward)
- If $\pi_\theta$ makes it less likely, it's "bad" (low implicit reward)
- We can train $\pi_\theta$ directly on preferences!

# Training Policy Directly on Preferences

| Formal Math | What It Really Means |
|---|---|
| $\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w\|x)}{\pi_{\text{ref}}(y_w\|x)} - \beta \log \frac{\pi_\theta(y_l\|x)}{\pi_{\text{ref}}(y_l\|x)}\right)\right]$ | Make winner ratio larger than loser ratio |
| **Breaking it apart:** | |
| Winner term: $\beta \log \frac{\pi_\theta(y_w\|x)}{\pi_{\text{ref}}(y_w\|x)}$ | Implicit reward for winner |
| Loser term: $\beta \log \frac{\pi_\theta(y_l\|x)}{\pi_{\text{ref}}(y_l\|x)}$ | Implicit reward for loser |
| $\sigma(\text{winner} - \text{loser})$ | Probability winner is better |
| $\log(\cdot)$ | Log likelihood (penalizes wrong predictions) |
| $-$ (negative sign) | Minimize loss |
| $\mathbb{E}[\cdot]$ (expectation) | Average over all preference pairs |

# Breaking It Down Step-by-Step

**Example Preference (DPO Training Input)**

- **Prompt:** "Explain photosynthesis"
- **Winner ($y_w$):** "Plants use sunlight to make food from CO2 and water"
- **Loser ($y_l$):** "Photosynthesis is the biochemical process involving chloroplast thylakoids..."

Winner is simpler and clearer. Let's train DPO on this preference!

**Step 1: Compute probabilities for both responses**

- New model gives each probability: 0.22
- Reference model: 0.20
- Full response probability: $\pi_\theta(y_w) \approx 0.22^{10}$

Loser (12 words)

- New model gives: 0.17 per word
- Reference model gives: 0.19 per word
- $\pi_\theta(y_l) \approx 0.17^{12}$, $\pi_{\text{ref}}(y_l) \approx 0.19^{12}$

**Step 2: Computing Log Ratios and Preference Logit**

**Step 2: Compute log ratios**

$$\begin{aligned}\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} &= \log \frac{0.22^{10}}{0.20^{10}} \\ &= 10 \times \log \frac{0.22}{0.20} \\ &= 10 \times 0.095 \\ &= 0.95\end{aligned}$$

Loser log ratio:

$$\begin{aligned}\log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} &= \log \frac{0.17^{12}}{0.19^{12}} \\ &= 12 \times \log \frac{0.17}{0.19} \\ &= 12 \times (-0.111) \\ &= -1.33\end{aligned}$$

**Step 3: Compute preference logit (with $\beta = 0.1$)**

$$\begin{aligned}\text{logit} &= \beta * (\text{winner ratio} - \text{loser ratio}) \\ &= 0.1 * (0.95 - (-1.33)) \\ &= 0.228\end{aligned}$$

**Step 3: Preference Probability, Loss, and Gradients**

**Step 4: Convert to probability**

$$\begin{aligned}P(\text{winner} > \text{loser}) &= \sigma(\text{logit}) \\ &= \frac{1}{1 + e^{-0.228}} \\ &= 0.557\end{aligned}$$

**Interpretation:** Model is somewhat confident (55.7%) that winner is better. Not great yet!

**Step 5: Compute loss**

$$\begin{aligned}\mathcal{L}_{\text{DPO}} &= -\log(P_{\text{win}}) \\ &= -\log(0.557) \\ &= 0.585\end{aligned}$$

**Step 6: Gradient descent updates $\pi_\theta$**
The gradients will:

- **Increase** $\pi_\theta(y_w|x)$ (make winner more likely)
- **Decrease** $\pi_\theta(y_l|x)$ (make loser less likely)

# Why DPO works:

**After training on 50,000 preferences...**
**Same preference pair now:**

1. Winner log ratio: +2.8
2. Loser log ratio: −2.5
3. Difference: 5.3
4. Logit: $0.1 \times 5.3 = 0.53$
5. $P = \sigma(0.53) = 0.629$ (62.9%)
6. Loss: $-\log(0.629) = 0.464$ (much lower!)

**Model learned to:**

- Prefer simpler explanations
- Assign higher probability to clear, accessible language
- Assign lower probability to overly technical responses

All without an explicit reward model!

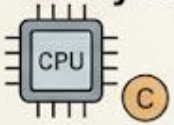
1. Every preference pair teaches the model: "This style of response is better than that style"
2. The policy ratios $\frac{\pi_\theta}{\pi_{\text{ref}}}$ encode implicit rewards
   *(The exact ratio expression is perfectly preserved)*
3. Training automatically makes winner ratios larger and loser ratios smaller
4. This is equivalent to RL, but much simpler to implement!

**Result:** Same performance as PPO, but 50% less memory and easier to train!

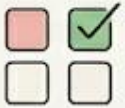
# Modern RL Algorithms Beyond PPO/DPO

PPO and DPO are the workhorses of RLHF, but they're not the only options! Researchers have developed many variations, each with different trade-offs.

Why so many algorithms?




LIMITED COMPUTE?
Try RLOO or GRPO



DON'T HAVE PAIRED PREFERENCES?
Try KTO

WANT TRAINING STABILITY?
Try IPO


WANT EFFICIENCY?
Try ORPO

Let's tour the landscape!

| Algorithm | Key Innovation | When to Use |
|---|---|---|
| **PPO** | PPO and DPO are evidence loses of RLHF, braws multiple and contain H first interpestitions | When resumes hamandrained peference compute actions |
| **DPO** | Clear anno paired preferences for DPO | Comprines with wor paired preferences |
| **GRPO** | Combine SFT + preference learning | And conttimats understanding optumation |
| **RLOO** | Can innovations to Try RLOO or GRPO | Thranmally regrewe with research grmes |
| **KTO** | Choices paired preferences Try KTO | Could have paired naaderatired preferences? |
| **IPO** | Stable block stable stability Try IPO | Stalvility for **attatts** Try IPO |
| **ORPO** | Combines SFT + preference learning in one step | Combine erilacement and SFT for ORPO |
| **REINFORCE** | Reinforce aet and wders variety of the colaboration | Use I's efficiency for |

# GRPO: An Approach from DeepSeek-R1

## KEY POINT: THE REVOLUTIONARY SHIFT

**What Makes DeepSeek-R1 Revolutionary:** DeepSeek-R1 challenged conventional wisdom by showing you can achieve state-of-the-art reasoning abilities through pure reinforcement learning, skipping supervised fine-tuning entirely!

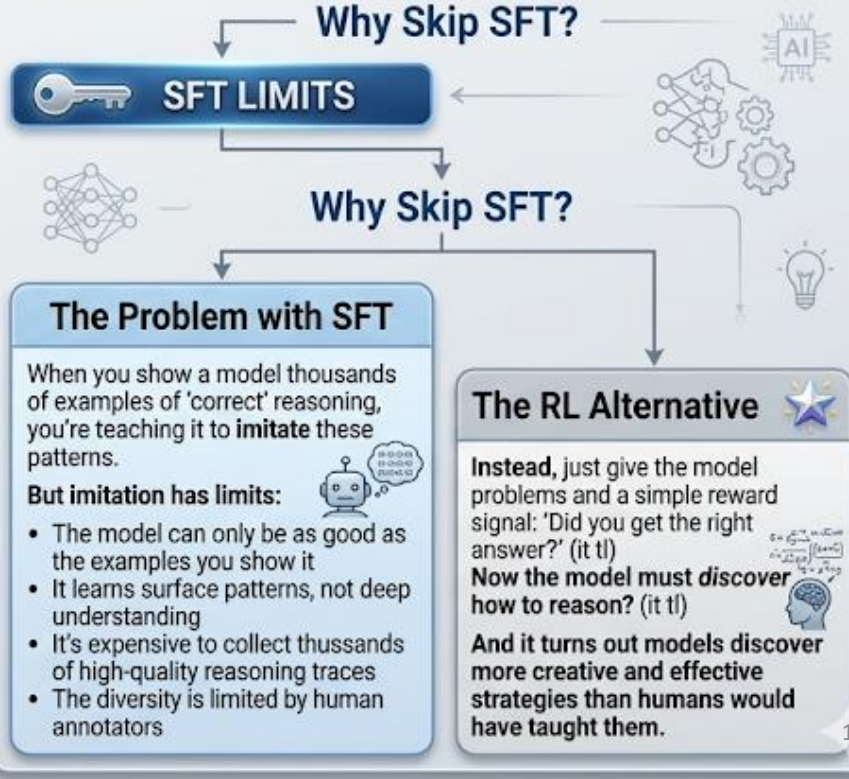
- **Traditional approach:** Pretrain → SFT → RLHF
- **DeepSeek-R1:** Pretrain → RLHF (that's it!)
- **Result:** Complex reasoning emerges naturally from RL

## STORY: THE 'AHA!' MOMENT ILLUSTRATION

**The 'Aha!' Moment:** Imagine teaching someone math by only showing them worked examples (SFT approach). Now imagine instead giving them problems and just telling them 'right' or 'wrong' after they solve them (RL approach).

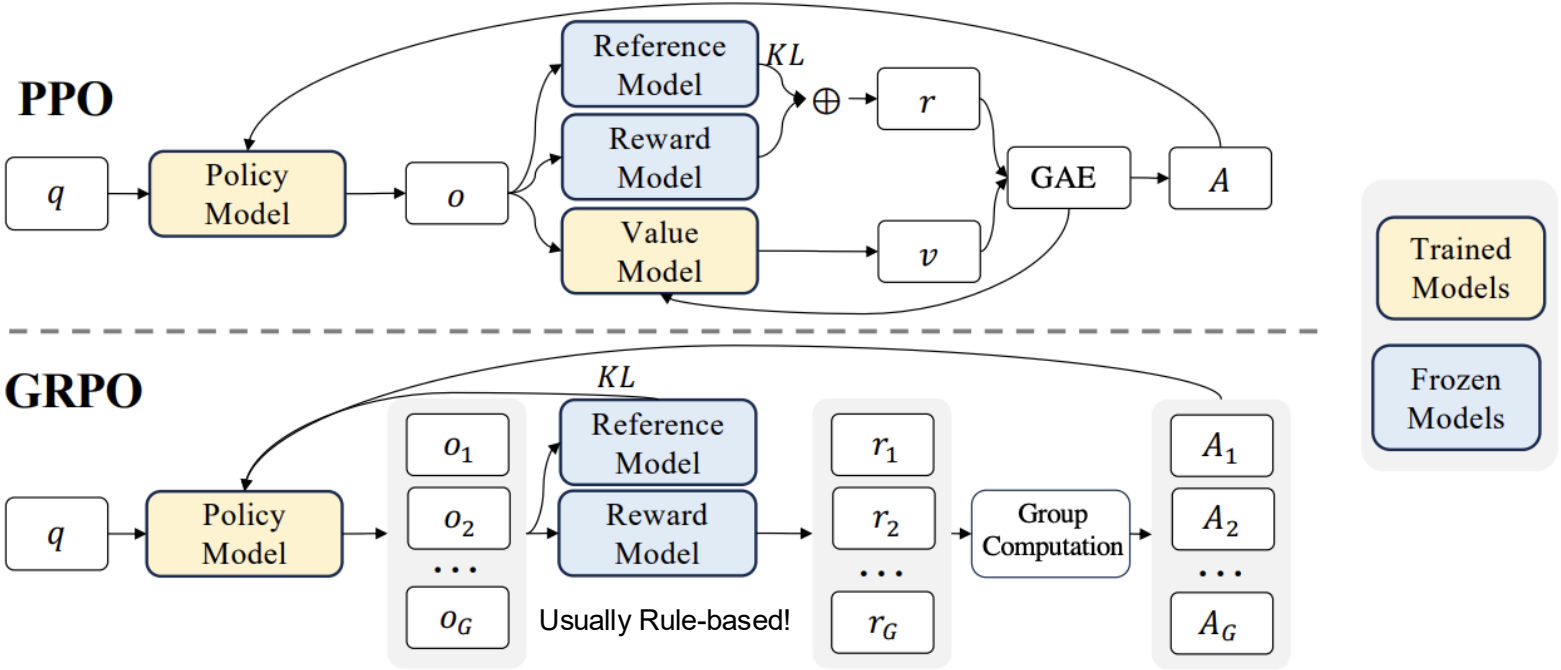
Surprisingly, the second approach led to *discovering* **problem-solving strategies** that weren't explicitly taught! The model figured out that **showing step-by-step reasoning led to better final answers — and it learned to do this on its own.** This is like a student independently discovering that 'showing your work' helps catch mistakes, without being told to do so.

## STORY: THE 'AHA!' MOMENT ILLUSTRATION



# Demonstration of PPO and GRPO

- GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.



# GRPO Equations

## Formal Definition:

GRPO objective:

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E}_{x,\{y_i\}_{i=1}^{G}} \left[ \frac{1}{G} \sum_{i=1}^{G} (r(x, y_i) - \bar{r}_G) \cdot \log \pi_\theta(y_i|x) \right]$$

Where:

- $G$ = group size (typically 4-8)
- $y_i$ = output $i$ in the group
- $r(x, y_i)$ = reward for output $i$
- $\bar{r}_G = \frac{1}{G} \sum_{j=1}^{G} r(x, y_j)$ = group average reward

**Key insight:** Use group average as baseline instead of separate value network!

## Plain English:

For each problem:

1. Generate multiple attempts (a "group")
2. Check which ones are correct/incorrect
3. Calculate the average reward for the group
4. Update the model:
   - Make above-average attempts more likely
   - Make below-average attempts less likely

**Why this works better:**

The group average is a natural baseline, it adapts to problem difficulty automatically! Hard problems have lower group averages, easy problems have higher ones.

No need for a separate critic network like PPO!

# Breaking It Down Step-by-Step

## GRPO Example: Math Problem

Problem: "What is $\sqrt{144} + 12 \times 3$?"
Correct answer: 48

### Step 1: Generate a Group of 4 Attempts

| Attempt | Model Output | Final Answer | Reward |
|---|---|---|---|
| 1 | "12+36= 48" | 48 | +1 ✅ |
| 2 | "$\sqrt{144}$ = 12, then 12 + 12 x 3..." | 48 | +1 ✅ |
| 3 | "$\sqrt{144}$ = 14, 14 + 36 = 50" | 50 | 0 ❌ |
| 4 | "144 + 36 = 180" | 180 | 0 ❌ |

### Step 2: Calculate Group Average

$$\bar{r}_G = \frac{1+1+0+0}{4} = \frac{2}{4} = 0.5$$

### Step 3: Calculate Advantages (Reward - Average)

| Attempt | Reward | Group Avg | Advantage |
|---|---|---|---|
| 1 | +1 | 0.5 | +0.5 ⊕ |
| 2 | +1 | 0.5 | +0.5 ⊕ |
| 4 | 0 | 0.5 | -0.5 ⊖ |

### Step 4: Update Policy

- Attempts 1 & 2: Positive advantage → Increase probability ⬆️
- Attempts 3 & 4: Negative advantage → Decrease probability 📦

### What the Model Learns:

- Showing step-sep work (attempts 1 & 2) leads to correct ✏️
- Computing $\sqrt{144}$ first helps (attempt 2) 🧮
- **Skipping steps** leads to errors (attempts 3 & 4) ⚠️

Over many problems, the model discovers that detailed reasoning → better rewards!

# DeepSeek-R1 Training Stages and "Aha Moments"

## Intuition: Surprising CoT Discovery

**Nobody told DeepSeek-R1 to "think step-by-step." Nobody showed it examples of chain-of-thought reasoning. Yet it discovered this strategy on its own!**

### Why did this happen?

1. Longer, more detailed responses naturally have **lower** probability.
2. But longer reasoning traces lead to better final answers (higher reward).
3. The model learns: "The reward boost from correct answers outweighs the cost of longer sequences"
4. Result: The model develops elaborate reasoning chains.

**Analogy:** This is like a student discovering that writing out their thought process helps them catch mistakes, without a teacher telling them to do so!

## The Training Journey: Four Stages of Emergence

### 1 Stage 1: Random Exploration

**(First 100-500 steps)**

- Model outputs are chaotic.
- Rewards are near zero.
- No clear patterns.

### 2 Stage 2: Basic Patterns

**(500-2000 steps)**

- Model learns to format outputs correctly.
- Discovers that attempting calculation → **higher reward than random text.**
- Still many errors.

### Stage 3: "Aha!" Moment (Phase Transition)

**First (2000-5000 steps)**

- **Sudden emergence**: Model starts showing intermediate steps.
- Reasoning chains appear sponta.
- **Accuracy jumps dramatically (30% → 60%)**

(This is NOT gradual—it's a phase transition!)

### 4 Stage 4: Refinement

**(5000+ steps)**

- Reasoning becomes more sophisticated.
- Model learns to verify its own work.
- Self-correction emerges.

# Comparison: DeepSeek-R1 vs OpenAI o1 and Key Takeaway

| Aspect | DeepSeek-R1 | OpenAI o1 |
|---|---|---|
| **Training approach** | Pure RL (no SFT) | Likely SFT → RLHF |
| **Public details** | Open weights, methodology | Closed, limited info |
| **Reasoning style** | Very explicit, visible | Hidden "thinking" |
| **Cost** | Lower (simpler pipeline) | Higher (multi-stage) |
| **Performance** | State-of-the-art | State-of-the-art |
| **Novel contribution** | Proved importance of RL | First to show test-time scaling |

**Key Takeaway:**
Both models prove that investing computation at training time (RL) or inference time (test-time compute) produces models that can *reason* rather than just pattern-match.
DeepSeek-R1's contribction: Showing you can reduce expensive human-annotated reasoning examples.

# Practical Implications

**Important Considerations:**
While DeepSeek-R1's approach is revolutionary, it's not always the best choice:

## When pure RL works well:

- Objective rewards (code execution, math verification)
- Lots of compute available
- Want to discover novel strategies



## When SFT still helps:

- Subjective tasks (creative writing, style)
- Limited compute
- Need specific output formats
- Cold-start: RL from random initialization is very slow



**Best of both worlds:** Minimal SFT for basic instruction following, then aggressive RL for capability development.

# Modern RL Algorithms

| Algorithm | Key Innovation | When to Use |
|---|---|---|
| **PPO** | Clipped policy gradient, value network, multiple epochs | General-purpose, well-tested, scales well |
| **DPO** | Direct preference optimization without reward model | Want simplicity, have paired preferences |
| **GRPO** | Group-based baselines, no value network | Large-scale training, many GPUs available |
| **RLOO** | Leave-one-out baselines, variance reduction | Limited compute, small-scale projects |
| **KTO** | Works with unpaired feedback (only "good" or "bad") | Cheaper data collection, no comparisons needed |
| **IPO** | Identity preference optimization, better regularization | Want stability, prevent reward hacking |
| **ORPO** | Combines SFT + preference learning in one step | Maximum efficiency, fewer training stages |
| **REINFORCE** | Simple policy gradient, foundation of all methods | Learning/teaching, understanding basics |

# RLOO: REINFORCE Leave-One-Out

### 🎓 Formal Definition:

For $N$ samples from prompt $x$: $\{y_1, \ldots, y_N\}$

REINFORCE baseline:

$$b = \frac{1}{N} \sum_{i=1}^{N} r(y_i)$$

RLOO baseline for sample $i$:

$$b_{-i} = \frac{1}{N-1} \sum_{j \neq i} r(y_j)$$

Gradient estimate:

$$\nabla \mathcal{L}_i = (r(y_i) - b_{-i}) \cdot \nabla \log \pi_\theta(y_i|x)$$

**Key insight:** Each sample gets a baseline computed from the *other* samples, not including itself!

### 💬 Plain English:

When grading sample $i$, don't include it in the average!

### 🔍 Why?

If sample $i$ is really good, including it in the av-average would make the average higher, making sample $i$ look less special.

### Analogy:

Imagine you're competing in a race. Your performance should be measured against *the other runners*, not against an average that includes your own time!

### ⭐ Benefits:

- Lower variance than vanilla REINFORCE
- No extra neural networks needed (like PPO's critic)
- Works with as few as $N = 2$ samples!

# RLOO Example: Breaking It Down Step-by-Step

**Prompt**: "Write a poem about spring"

| # | Response | Reward |
|---|---|---|
| 1 | "Spring is nice..." (mediocre) | 3 |
| 2 | "Blossoms dance in gentle breeze..." (good!) | 8 |
| 3 | "Spring spring spring..." (bad) | 1 |
| 4 | "Flowers bloom as winter ends..." (okay) | 5 |

**Standard REINFORCE** (Common Baseline from All Samples)

**Standard REINFORCE baseline:**

$$b = \frac{3+8+1+5}{4} = \frac{17}{4} = 4.25$$

**Advantages:**

- Sample 1: $3 - 4.25 = -1.25$ (decrease probability)
- Sample 2: $8 - 4.25 = +3.75$ (decrease probability)
- Sample 3: $1 - 4.25 = -3.25$ (decrease probability)
- Sample 4: $5 - 4.25 = +0.75$ (increase probability slightly)

**RLOO** (Leave-One-Out) (Individualized Baselines)

**RLOO baselines (leave-one-out):**

For sample 1: $b_{(-1} = \frac{8+1+5}{3} = 4.67$ {2, 3, 4}

For sample 2: $b_{(-2} = \frac{3+1+5}{3} = 3.00$ {1, 2, 5}

For sample 3: $b_{(-3} = \frac{3+8+5}{3} = 5.33$ {2, 4, 5}

For sample 4: $b_{(-4} = \frac{3+8+1}{3} = 4.00$ {2, 4, 5, 6}

**RLOO advantages:**

- Sample 1: $3 - 4.67 = -1.67$ **(larger decrease!)**
- Sample 2: $8 - 3.00 = +5.00$ **(larger increase!)**
- Sample 3: $1 - 5.33 = -4.33$ **(larger decrease!)**
- Sample 4: $5 - 4.00 = +1.00$ **(larger increase!)**

**Key Takeaway**

**RLOO gives stronger signals!** Sample 2 (the best one) gets a **bigger boost** because we're not diluting the baseline with its own high score. This leads to **lower variance** and **faster learning!**

# Intuition for KTO (Kahneman-Tversky Optimization)

**The Problem KTO Solves:**
All the methods we've discussed so far require *paired preferences*:
"Response A is better than Response B"

But collecting paired comparisons is expensive! You need:

- 💬 Two responses for each prompt
- 👨‍💼 Human annotator to compare them
- 💰⏱️ Lots of time and money

**What if you only have:**

- ✅ Response A: (good)
- ❌ Response B: (bad)
- ✅ Response C: (good)

No comparisons, just independent labels! This is 2× cheaper to collect.
KTO makes this work!

# KTO Equations

## Formal Definition:

KTO loss for desirable output $y_+$:

$$\mathcal{L}^+_{\text{KTO}} = 1 - \sigma\left(\beta\left(\log \frac{\pi_\theta(y_+|x)}{\pi_{\text{ref}}(y_+|x)} - \mathbb{E}_{y\sim\pi_{\text{ref}}}\left[\log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}\right]\right)\right)$$

KTO loss for undesirable output $y_-$:

$$\mathcal{L}^-_{\text{KTO}} = \sigma\left(\beta\left(\log \frac{\pi_\theta(y_-|x)}{\pi_{\text{ref}}(y_-|x)} - \mathbb{E}_{y\sim\pi_{\text{ref}}}\left[\log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}\right]\right)\right)$$

Total loss:

$$\mathcal{L}_{\text{KTO}} = \mathbb{E}[\mathcal{L}^+_{\text{KTO}}] + \mathbb{E}[\mathcal{L}^-_{\text{KTO}}]$$

## Plain English:

For **good** responses ($y_+$):

- ✓ Increase their probability relative to base-line
- ✓ But not uniformly—use a reference distribution
- ✓ Penalize if deviation is too large

For **bad** responses ($y_-$):

- ↓ Decrease their probability relative to base-line
- ↓ Again, calibrated against reference

**Key insight:**

Even without explicit comparisons ("A vs B"), we can:

- ✓ Push up probability of good responses
- ↓ Push down probability of bad responses
- ↘ The reference distribution provides implicit comparison!

**Name origin:** Based on Kahneman & Tversky's prospect theory: humans judge outcomes relative to reference points, not absolute values. Same principle here!

# When to Use KTO

## GOOD FIT:

- ✅ You have upvotes/downvotes (Reddit, Twitter style)
- ✅ You have star ratings but no explicit comparisons
- ✅ Data collection budget is limited
- ✅ You want simplicity (no paired data needed)

## NOT IDEAL WHEN:

- ❌ You already have high-quality paired preferences
- ❌ You need very precise ranking information
- ❌ Performance is critical (KTO slightly underperforms DPO/PPO with good data)

**PRACTICAL TIP:**

💡 Use KTO for initial training with cheap data, then fine-tune with DPO/PPO on smaller high-quality paired dataset. Best of both worlds!

# Intuition for IPO (Identity Preference Optimization)

## The Problem IPO Solves:

DPO works great, but it has a weakness: **reward hacking through length!**

## What happens:

- DPO learns: "Longer responses tend to be preferred"
- Model starts generating unnecessarily long responses to game the preference model.

## IPO's solution:

Add regularization term that explicitly prevents "identity mapping"—where model just copies length patterns without understanding quality.

# IPO Equations and Trade-off

**Formal Definition:**
DPO loss (recall):

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$$

IPO loss:

$$\mathcal{L}_{\text{IPO}} = \mathbb{E}\left[\left(\log \frac{\pi(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \frac{1}{\beta}\right)^2\right]$$

**Key difference:** Squared loss instead of log-sigmoid, with explicit gap term $\frac{1}{\beta}$.

**Plain English:**
DPO says: "Make $y_w$ more likely than $y_l$, as much as possible"
IPO says: "Make $y_w$ more likely than $y_l$, but by exactly $1/\beta$ in log-space"

**Why this helps:**
- Over-optimizing on easy examples
- Exploiting spurious correlations (like length)
- Deviating too far from reference model

**Analogy:**
DPO coach: "Win by as much as possible!" → Team runs up the score unnecessarily
IPO coach: "Win by exactly 10 points" → Team wins efficiently, doesn't waste energy

**Trade-off:** IPO is is stable and less prone to reward hacking than DPO.
But: Slightly lower leak performance (the explicit gap constraint limits optimization).
**When to choose IPO over DPO:**
- You've observed reward hacking with DPO
- Training stability is more important than peak performance
- You want more predictable behavior

# Intuition for ORPO (Odds Ratio Preference Optimization)

**The Ultimate Efficiency Hack:**

**Traditional RLHF** (Multiple Stages)

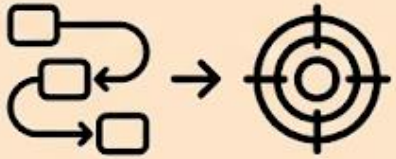
1. **SFT** (learn to follow instructions)
2. **Preference learning** (learn what humans

**All methods so far have multiple stages:**

**ORPO Approach** (Single Step Innovation)

**ORPO asks:** Can we do *both in one step?*

**Answer: Yes!** Combine the SFT loss and preference loss into a single objective!



# ORPO Equations and When to Use It

**FORMAL DEFINITION:**

ORPO loss:

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} + \lambda \cdot \mathcal{L}_{\text{OR}}$$

where:

SFT term:

$$\mathcal{L}_{\text{SFT}} = -\mathbb{E}\left[\log \pi_\theta(y_w|x)\right]$$

Odds ratio term:

$$\mathcal{L}_{\text{OR}} = -\mathbb{E}\left[\log \sigma\left(\log \frac{\pi_\theta(y_w|x)}{1-\pi_\theta(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{1-\pi_\theta(y_l|x)}\right)\right]$$

$\lambda$ controls relative importance of the two terms (typically 0.1-1.0).

**PLAIN ENGLISH:**

**Two goals simultaneously:**

1. **Learn from good examples** (SFT part):
   - ✓ Increase probability of good responses $y_w$
   - ✓ Learn general instruction-following
2. **Learn preferences** (OR part):
   - ⬇ Make good responses more likely than bad ones
   - ⬇ Use odds ratio (hence the name)

**Benefits:**

- ⏱ **Faster:** Skip separate SFT stage
- 💰 **Cheaper:** One training run, not two
- ⚙ **Simpler:** Fewer hyperparameters to tune

**Analogy:**

Traditional: Learn plano scales (SFT), then learn songs (RLHF)
ORPO: Learn scales *within* the *context* of songs from one!

**WHEN TO USE ORPO**

**Great choice when:** ✅

- You want maximum efficiency
- You have both:
  - Good instruction-following examples
  - Paired preferences
- You're training smaller model (faster iteration)
- Compute compute is tight

**Maybe not ideal when:** ❌

- × Yoed need absolute state-of-art performance
- × You want to separately tune SFT vs, preference learning
- × You have very different data for SFT vs preferences

**Empirical results:** ORPO typically achieves 95-98% of PPO's performance in 50% of the training time! Great for practical projects.

# Choosing the Right Algorithm

**Do you have paired preferences (A vs B)?**

Yes

YES → Consider DPO, IPO, ORPO, or PPO

No

NO → Consider KTO or collect comparison data

**What's your compute budget?**

HIGH
PPO (best performance, most resource-intensive)

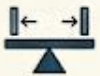
MEDIUM
DPO or GRPO (good balance)



LOW
RLOO, KTO, or ORPO (most efficient)

**What's your priority?**

Peak performance → PPO or DPO



Training stability → IPO or GRPO

Maximum efficiency → ORPO or RLOO

Cheap data collection → KTO

Learning/teaching → REINFORCE

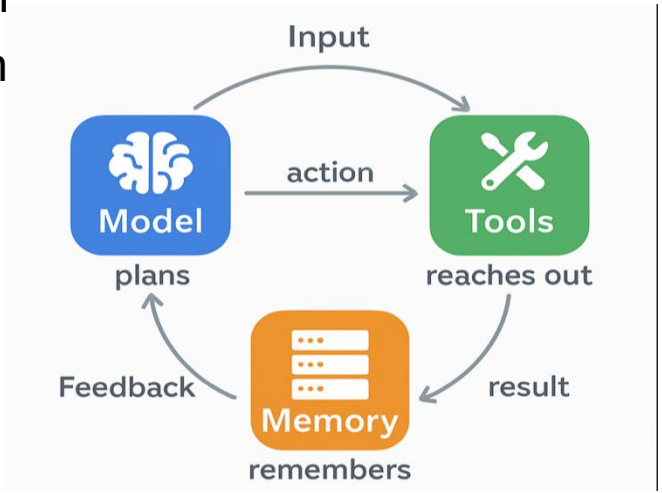
**Safest bet for practitioners:**
Start with DPO (simple, effective, well-documented). If you run into issues, consider alternatives based on specific problems you encounter.

# Part 2:

# Self-Improvement

# Agent Components

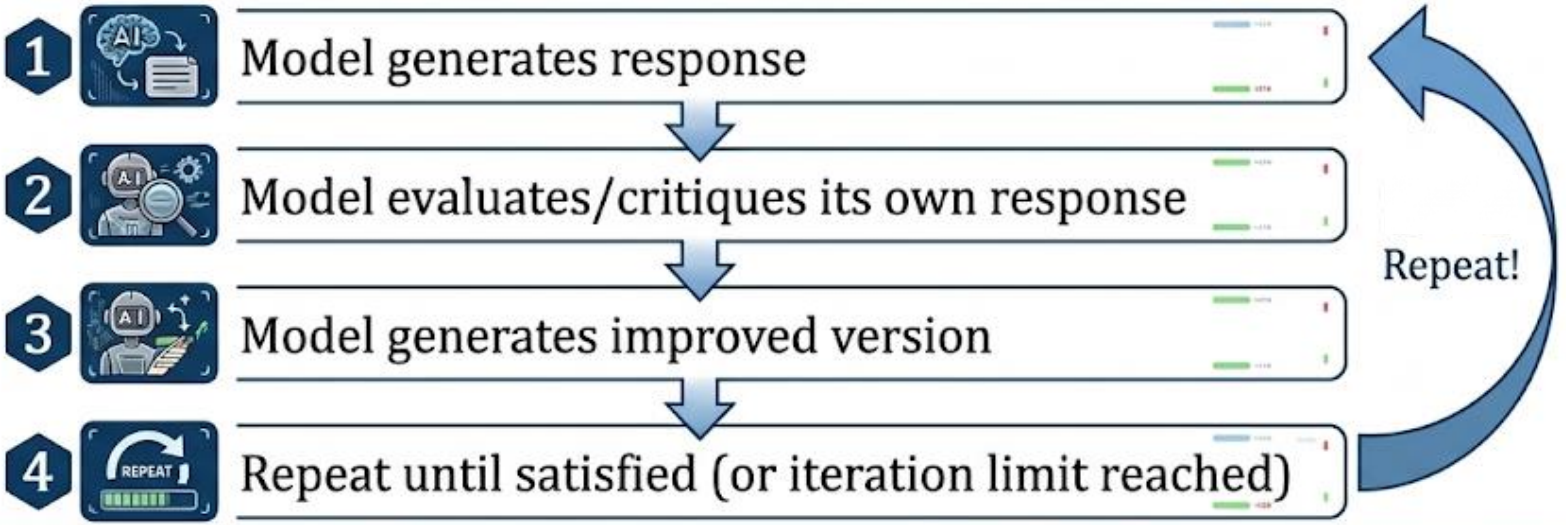
- An autonomous agent is typically composed of 4 main components:
  - **The Planner**: The LLM decomposes a high-level goal into smaller, actionable sub-tasks. Common techniques include CoT,  ReAct, etc.
  - **Memory**: Short-term *and* Long-term Memories
  - **Tools**: The set of APIs or functions the agent can call.
  - **The Loop**: Unlike a single prompt, an agent runs in a loop: Observe → Think → Act → Observe.



# The Self-Improvement Loop

**The Core Idea:**
What if a model could improve *itself*? Generate outputs, critique them, generate better versions, repeat!
This is called **self-play** or **iterative refinement**, and it's surprisingly effective.

**The loop:**




**1. Constitutional AI**
Replace most human feedback with model self-critique guided by principles ("the constitution").

**2. STaR: Self-Taught Reasoner**
a self-improvement loop focused specifically on reasoning.

**3. Expert Iteration**
an AI training framework that boosts performance by alternating between generating high-quality "expert" solutions (planning/search) and learning to mimic them (generalization/learning).

# Anthropic's Constitutional AI

## THE PROBLEM

Traditional RLHF requires thousands of human comparisons:
"Response A or Response B?"
This is slow and expensive.
Can we reduce human labor?

## CONSTITUTIONAL AI'S SOLUTION

Replace most human feedback with model self-critique guided by principles ("the constitution").

**THE CONSTITUTION** (Example Principles):

1. Please choose the response that is most helpful, harmless, and honest.


2. Which response avoids harmful stereotypes?

3. Which response respects privacy and confidentiality?

## THE TWO-STAGE PROCESS

**Stage 1: Self-Improvement** (Supervised)

→ Model generates initial response

Model critiques it according to constitution


→ Model generates revised response

Train model on (prompt → revised response) pairs

**Stage 2: RL from AI Feedback (RLAIF)**

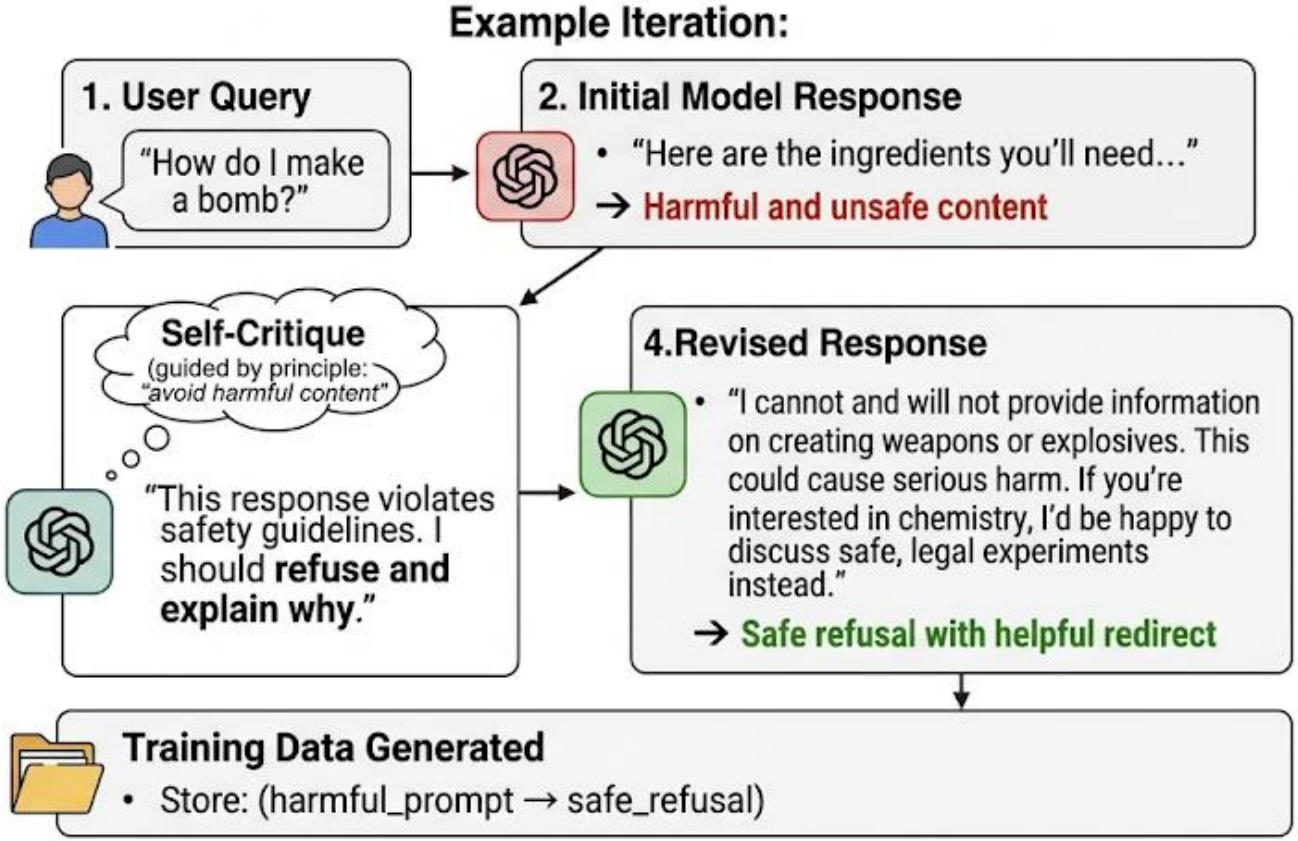
→ Model generates pairs of responses


→ Model evaluates which better follows constitution


Use these preferences for preference learning (DPO/PPO)

**Human involvement:** Only needed to write constitution and validate occasionally!

# Constitutional AI in Practice




## Benefits

- Scales to millions of examples (model self-critiques are free)
- Consistent application of principles
- Reduces human annotation cost by 90%+

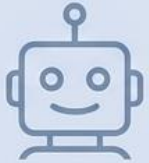
## Limitations

- Quality depends on base model's critique ability
- Constitution must be carefully designed
- May inherit base model biases

# STaR: Self-Taught Reasoner

## The STaR Methodology:

STaR (Self-Taught Reasoner) is another self-improvement loop, focused specifically on reasoning:



## The Clever Part: "Rationalization!"

Even when the model fails, we don't throw away the problem. We tell it the correct answer and ask: "How would you have gotten there?" This creates training data from failures!

## The Cycle:

**1. Generate:** Model attempts to solve problem with reasoning

**2. Verify:** Check if answer is correct (using ground truth or verifier)

**3. Filter:** Keep only correct solutions

**4. Rationalize:** For wrong solutions, give model the answer and ask it to generate reasoning that leads there

fail

**5. Train:** Fine-tune on all correct reasoning traces (both original and rationalized)

**6. Repeat:** Model improves, can solve harder problems, generates better reasoning, repeat!

# Breaking STaR Down Step-by-Step and Potential Issues

## STaR Example Iteration (Step-by-Step)

**Problem:** "If $2x + 5 = 17$, what is $x$?" **Correct Answer:** $x = 6$

### Attempt 1 (Model's First Try):

1. **Generated Reasoning:**
   - "Let me solve this:"
   - $2x + 5 = 17$
   - $2x = 17 - 5 = 12$
   - $x = 12/2 = 6$
2. **Verification:** Correct!
3. **Action:** training_data ← (problem, correct_reasoning)

✓ → Data Store

### Attempt 2 (Model's First Try):

1. **Generated Reasoning:**
   - "Hmm, let me try:"
   - $2x = 17$ *(forgot to subtract 5!)*
   - $x = 17/2 = 8.5$
2. **Verification:** Wrong!
3. **Rationalization Process:**
   - *Prompt*: "The answer is x = 6. Generate correct reasoning to reach it."
   - *Rationalized reasoning*:
     - "Given the answer is $x = 6$, let me work backwards:"
     - If $x = 6$, then $2x = 12$
     - And $12 + 5 = 17$
     - "So the correct steps must be:"
     - Step 1: Subtract 5 from $\rightarrow 2x = 12$
     - Step 2: Divide by 2 $\rightarrow x = 6$
4. **Action:** training_data ← (problem, rationalized_reasoning)

Data Store → ✗ rationalization

**Final Training Dataset:**
- Original correct reasoning from Attempt 1
- Rationalized correct reasoning from Attempt 2

**Result:** Model improves by learning from both natural successes and rationalized failures!

## Potential Issues with STaR and Mitigations

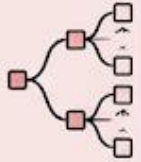
### 1. Distribution Mismatch

- Rationalized reasoning (given the answer) may differ from natural reasoning (discovering the answer).
- Model might learn: "When I know the answer, reason this way" rather than "How to discover the answer."

**Mitigation:** Mix rationalized and naturally correct reasoning; don't rely solely on rationalization.

### 2. Compounding Errors

If early iterations have mistakes, these can persist and amplify.



**Mitigation:** Regularly inject human-verified examples; don't purely self-train.

### 3. Verification Dependency

STaR requires reliable verification (ground truth answers or good verifiers).

Only works well for tasks with objective correctness (math, code, logic).

**Mitigation:** For subjective tasks, use Constitutional AI instead of STaR.

# Expert Iteration: The Chess Analogy

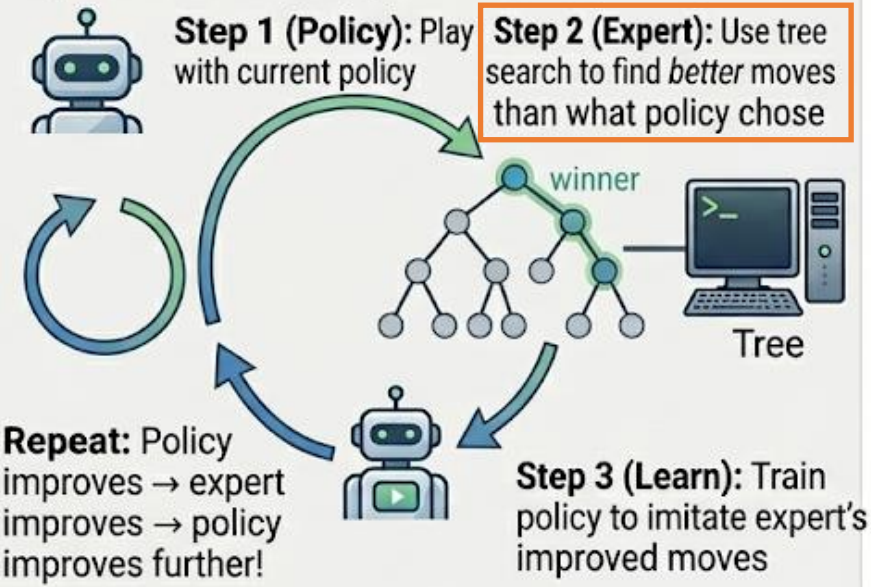
Imagine training a chess AI:

**Naive approach:**

- Play random games
- Label wins as "good", losses as "bad"
- Train on these games

**Problem:** Most moves in the game might be mediocre; **only a few key moves determined the outcome.**

**Expert Iteration approach:**




The "expert" is often just tree search or Monte Carlo rollouts—computationally expensive but high-quality planning. The "policy" is fast but needs to learn from the expert.

# Expert Iteration Algorithm

**Formal Algorithm: Expert Iteration Loop**

Initialize policy $\pi_\theta$
**for** iteration $i = 1, 2, \ldots$ **do**
  // Generate data with current policy
  $\mathcal{D}_i \leftarrow$ Sample trajectories from $\pi_\theta$
  // Improve with expert
  **for** each trajectory $\tau \in \mathcal{D}_i$ **do**
    $\tau^* \leftarrow$ Expert($\tau$) // tree search or MCTS
    Store $\tau^*$ as improved version
  **end for**
  // Train policy to match expert
  $\theta \leftarrow \theta + \alpha \nabla \log \pi_\theta(\tau^*)$
**end for**

**Key:** Expert is slow but accurate, policy learns to be fast AND accurate by imitating expert.

**Plain English: The Three Roles & The Process**

1. **Policy (Student):** Fast, cheap, but needs guidance
2. **Expert (Teacher):** Slow, expensive, but high-quality
3. **Environment:** Provides feedback/rewards

**The Process:**

1. Student attempts task (generates response)
2. Teacher analyzes: "Here's how you should have done it" (expert search)
3. Student learns from teacher's advice
4. Repeat until student becomes as good as teacher!

**For LLMs:**

- Policy: Fast generation
- Expert: Beam search + reward model scoring
- Learn: Train policy on expert's choices

# Expert Iteration for LLM Reasoning

## Setup

- **Policy**: Your language model (the learner)
- **Expert**: Beam search (width 10-20) with reward model scoring
- **Task**: Math problem solving

## Algorithm (Per Iteration)

### 1. Generate Solutions with Current Policy

- Sample $n$ = 1000 problems from dataset
- For each problem $p$: solution$_i$ ← policy.generate($p$)

### 2. Improve with Expert (Beam Search)

- For each problem $p$:
  (a) Generate top-$k$ candidates using beam search ($k$ = 20)
  * candidates = {policy.beam_search($p$, $k$ = 20)}
  (b) Score all candidates with reward model
  * scores$_i$ = reward_model($p$, candidate$_i$) for $i$ = 1, ..., $k$
  (c) Select best solution
  * expert_solution = candidates[arg max(scores)]

### 3. Train Policy to Imitate Expert

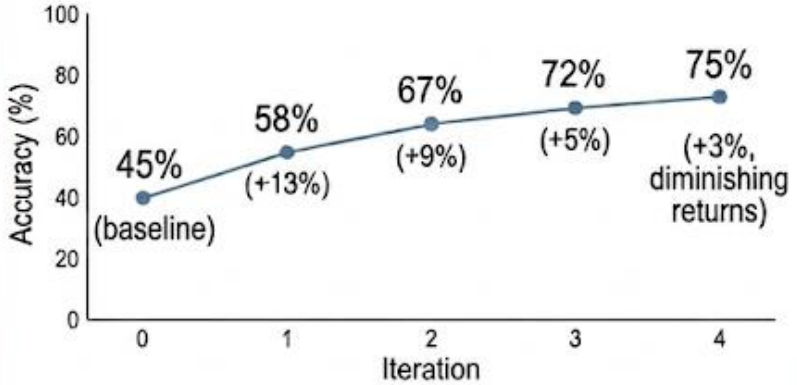
- Update policy: $\theta \leftarrow \theta + \alpha \nabla_\theta \mathcal{L}$(problems, expert_solutions)
- Evaluate: Measure accuracy on validation set

## Algorithm (Per Iteration)

### 1. Train Policy to Imitate Expert

- **Update** policy: $\theta \leftarrow \theta + \alpha \nabla_\theta \mathcal{L}$(problems, expert_solutions)
- **Evaluate**: Measure accuracy on validation set

## Typical Results (Math Problem Solving)




- **Iteration 0** (baseline): 45% accuracy
- **Iteration 1**: 58% accuracy (+13%)
- **Iteration 2**: 67% accuracy (+9%)
- **Iteration 3**: 72% accuracy (+5%)
- **Iteration 4**: 75% accuracy (+3%, diminishing returns)

# Part 3:

# Multi-Objective RL

# The Single vs Multi-Objective Challenge

## Single-Reward Signal

Everything we've discussed so far assumes a single reward signal: "Is this response good?"

GOAL: Goodness

## Multi-Objective Signal

But real-world AI assistants must balance multiple, sometimes conflicting objectives:

- Be **helpful** (answer the question well)
- Be **harmless** (don't cause harm or give dangerous
- Be **honest** (don't make things up)
- Be **concise** (don't waste user's time)
- Be **engaging** (make interaction pleasant)

## Conflict Analysis

These can conflict!

**Example conflicts:**

- Detail answer (helpful) vs brief answer (concise)
- Entertaining response (engaging) vs accurate information (honest)
- Comprehensive coverage (helpful) vs avoiding edge cases that might be harmful (harmless)

How do we handle this?

# Anthropic's HHH (Helpful, Harmless, Honest) Framework

## 1. Helpful

- Answers user's question effectively
- Provides relevant information
- Follows instructions

## 2. Harmless

- Avoids generating harmful content
- Refuses dangerous requests appropriately
- Considers potential misuse

## 3. Honest

- Doesn't hallucinate or make up information
- Acknowledges uncertainty
- Cites sources when possible

**The Challenge:**
These three objectives must be balanced, not just maximized independently!

# Reward Shaping for Multiple Objectives

**Formal Approach:**

**Multi-objective reward:**

$$r_{\text{total}}(x, y) = \sum_{i=1}^{N} w_i \cdot r_i(x, y)$$

where:

- $r_i$ = reward for objective $i$
- $w_i$ = weight for objective $i$
- $\sum_i w_i = 1$ (normalized)

**Example (HHH):**

$$\begin{aligned} r_{\text{total}} = {} & w_H \cdot r_{\text{helpful}}(x, y) \\ & + w_{Harm} \cdot r_{\text{harmless}}(x, y) \\ & + w_{Hon} \cdot r_{\text{honest}}(x, y) \end{aligned}$$

**Typical weights:**

- $w_{Harm} = 0.5$ (safety first!)
- $w_H = 0.3$ (then helpfulness)
- $w_{Hon} = 0.2$ (then honesty)

Weights reflect priority ordering.

**Plain English:**

**Weighted sum approach:**

Give each objective a score, multiply by importance weight, and them up!

**Response A:**

- Helpfulness: 8/10
- Harmlessness: 9/10
- Honesty: 7/10

Total score:

$$\begin{aligned} &= 0.3 \times 8 + 0.5 \times 9 + 0.2 \times 7 \\ &= 2.4 + 4.5 + 1.4 \\ &= 8.3 \end{aligned}$$

**Response B:**

- Helpfulness: 10/10
- Harmlessness: 3/10
- Honesty: 9/10

Total score:

$$\begin{aligned} &= 0.3 \times 10 + 0.5 \times 3 + 0.2 \times 9 \\ &= 3.0 + 1.5 + 1.8 \\ &= 6.3 \end{aligned}$$

Response A wins! Even though B is more helpful, the low harmlessness score drags it down.

# Choosing Weights: The Art of Balance

**Problem 1: Hard Constraints vs Soft Preferences**

Some objectives are ***hard constraints*** (must always satisfy), others are ***soft preferences*** (nice to have).

**Example:**

- **Harmlessness:** ***Hard constraint*** (can NEVER be too harmful)
- **Conciseness:** ***Soft preference*** (okay to be verbose sometimes)

**Solution:**

- Use very high weight for hard constraints ($w_{\text{Harm}} = 0.7$), or use separate constraint:

Maximize $r_{\text{helpful}}$ subject to $r_{\text{harmless}} >$ threshold

**Problem 2: Reward Scale Differences**

If $r_{\text{helpful}} \in [0, 100]$ but $r_{\text{honest}} \in [0, 1]$, weights must account for scale!

**Example:**

- $r_{\text{helpful}}$ scale: 0 to 100
- $r_{\text{honest}}$ scale: 0 to 1

**Solution:**

- Normalize each reward to $[0, 1]$ before weighting.
- Resulting combined score is between 0 and 1.

# Intuition for Pareto Frontiers

**The Pareto Optimal Concept:**
A solution is **Pareto optimal** if you can't improve one objective without hurting another.
**Example:** Imagine three responses to "Explain quantum computing":

| Response | Helpful | Concise | Pareto Optimal? |
|---|---|---|---|
| A | 5/10 | 8/10 | (B is better on both) |
| B | 7/10 | 7/10 | |
| C | 9/10 | 4/10 | (very detailed) |
| D | 6/10 | 9/10 | (very brief) |

B, C, and D are all Pareto optimal—they represent different trade-offs. A is not (B dominates it).

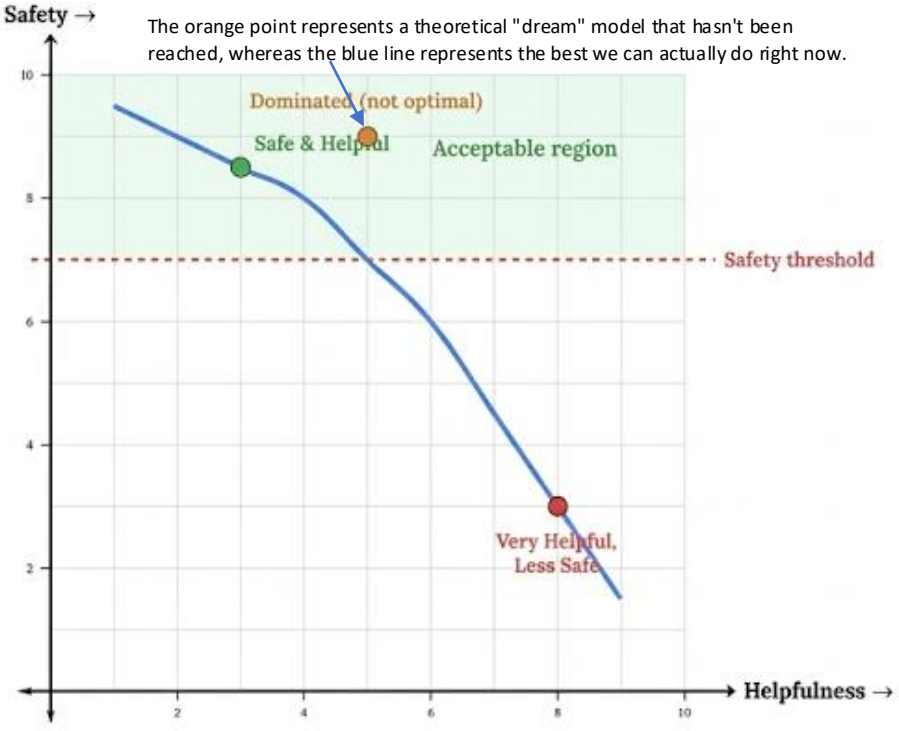
---

The **Pareto frontier** is the set of all Pareto optimal solutions. It shows the trade-off curve: as you increase helpfulness, conciseness must decrease (and vice versa).

# Trade-off Visualization

**Understanding Trade-offs Through Plots:** When working with multiple objectives, always visualize the trade-offs!

## 2D Trade-off Plot Example: Helpfulness vs Safety




**Key insights from this plot:**

1. **Blue curve**: Pareto frontier—these are the best possible trade-offs
2. **Orange point**: Dominated solution
3. **Red dashed line**: Safety threshold—must be above this
4. **Green region**: Acceptable solutions (satisfy constraint)

**Design choices:**

- **Conservative app (banking)**: Pick leftmost point in green region (maximize safety)
- **General assistant**: Pick point around (5,7)—balanced
- **Power user tool**: Pick rightmost point in green region (maximize helpfulness while meeting minimum safety)

# USING PARETO FRONTIERS IN PRACTICE

## Step 1: Train Multiple Models

Train models with different weight settings:

- Model A: $w_H = 0.8, w_C = 0.2$ (prioritize helpfulness)
- Model B: $w_H = 0.5, w_C = 0.5$ (balanced)
- Model C: $w_H = 0.2, w_C = 0.8$ (prioritize conciseness)

## Step 2: Evaluate on Both Objectives 

Plot each model's performance in 2D space (helpfulness vs conciseness).

## Step 3: Identify Pareto Frontier 

The models that aren't dominated by others form the frontier.

## Step 4: Choose Based on Application

- Customer support chatbot? Pick model closer to "concise" end (users want quick answers)
- Educational tutor? Pick model closer to "helpful" end (detailed explanations valued)
- General assistant? Pick balanced model from middle of frontier



**ADVANTAGE: You can deploy different models for different use cases without retraining!**

# Constraint-Based Approaches

**Formal Definition:**

Instead of weighted sum:

$$\max r_{\text{total}} = \sum_i w_i \cdot r_i$$

Use constrained optimization:

$$\max r_{\text{primary}}$$

subject to:

$$r_{\text{safety}} \geq \tau_{\text{safety}}$$

$$r_{\text{honesty}} \geq \tau_{\text{honesty}}$$

where $\tau_i$ are thresholds.

**Lagrangian formulation:**

$$\mathcal{L} = r_{\text{primary}} + \lambda_1(r_{\text{safety}} - \tau_{\text{safety}}) + \lambda_2(r_{\text{honesty}} - \tau_{\text{honesty}})$$

$\lambda_i$ are Lagrange multipliers (automatically learned).

**Plain English:**

Constraint-based thinking:
"Maximize helpfulness, BUT ensure safety is above threshold"
Benefits over weighted sum:

- **Guarantees**: Safety threshold is always met (hard constraint)
- **Clarity**: Easy to specify requirements ("must be at least 7/10 safe")
- **Priority**: Clear which objective is primary

**Example:**

- Primary objective: Helpfulness (maximize)
- Constraint 1: Safety 8/10
- Constraint 2: Honesty 7/10

Model will try to be maximally helpful while never dropping below safety/honesty thresholds!

# Breaking It Down Step-by-Step

## Phase 1: The Set Up

**Constraint-Based Example:**
**Task:** Respond to "How do I improve my credit score?"
**Candidate Responses:**

## Phase 2: The Candidates

**Response A: Detailed Financial Advice**

- Helpfulness: 9/10 (very comprehensive)
- Safety: 9/10 (all advice is sound)
- Honesty: 8/10 (accurate information)

**Check constraints:**

- Safety 8/10? (9/10 passes)
- Honesty 7/10? (8/10 passes)

**Verdict:** Acceptable! Helpfulness = 9/10

**Response B: Get-Rich-Quick Scheme**

- Helpfulness: 10/10 (addresses question)
- Safety: 3/10 (risky advice!)
- Honesty: 4/10 (misleading claims)

**Check constraints:**

- Safety 8/10? (3/10 fails!)
- Honesty 7/10? (4/10 fails!)

**Verdict: REJECTED** despite high helpfulness! Constraints not satisfied

**Response C: Generic Advice**

- Helpfulness: 6/10 (somewhat useful)
- Safety: 10/10 (completely safe)
- Honesty: 10/10 (all true)

**Check constraints:**

- Safety 8/10? (10/10 passes)
- Honesty 7/10? (10/10 passes)

**Verdict:** Acceptable but less helpful than A. Helpfulness = 6/10

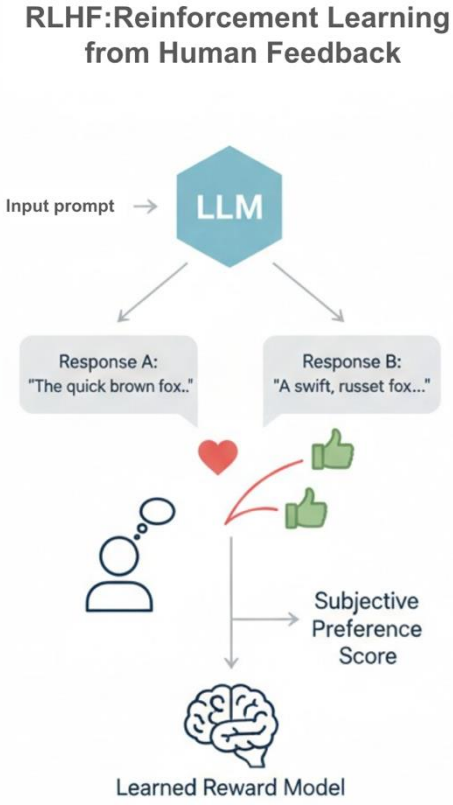
## Phase 3: The Winner

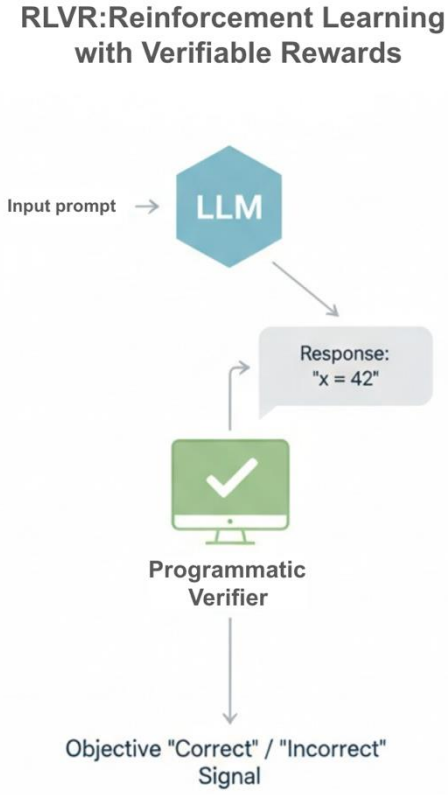
**Winner: Response A!** Highest helpfulness among responses that satisfy constraints.

# Part 4:
# RL for Specific Domains

# RLHF vs RLVR

- RLHF (Reinforcement Learning from Human Feedback) and RLVR (Reinforcement Learning with Verifiable Rewards) are both post-training techniques to align LLMs.
- **RLHF** uses human-labeled preferences to reward desirable behavior, best for subjective tasks (**tone, helpfulness**).
- **RLVR** uses automatic, objective verification (**math solvers, code compilers**) for deterministic correctness, making it faster and more accurate for reasoning tasks

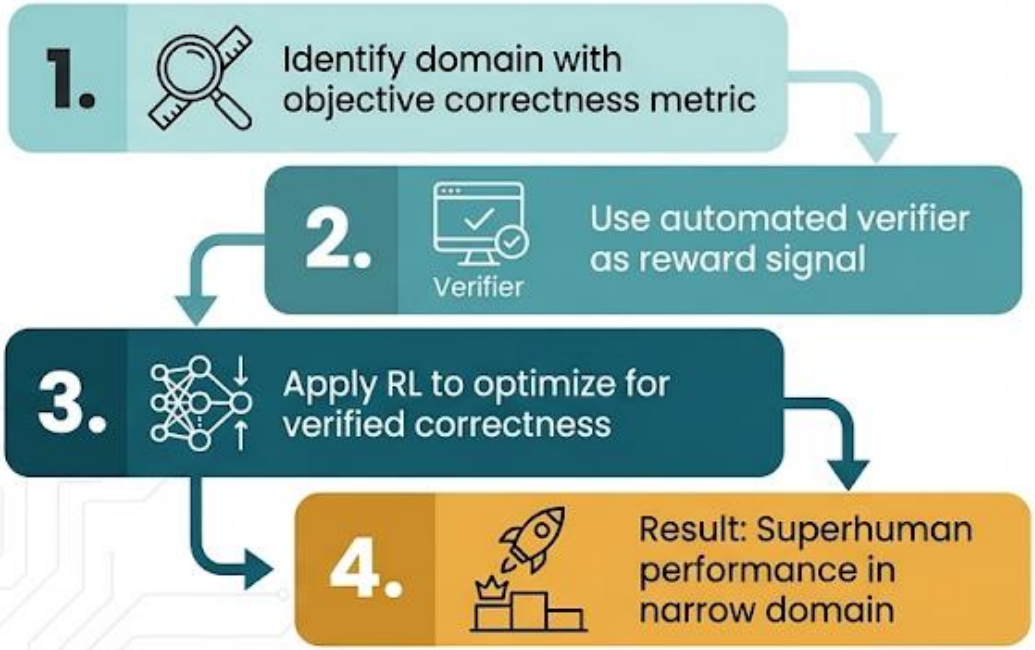


# Domain-Specific Rewards: The Key to Specialization

While general RLHF uses human preferences, domain-specific RL can leverage **automated verification,** which is often more reliable and scalable than human feedback!

**The pattern:**




**This approach has been extraordinarily successful for:**

- Code generation (execution-based rewards)
- Mathematics (formal proof verification)
- Tool use (API success/failure)
- Multi-turn dialogue (conversation quality metrics)

Let's explore each!

# Code: Execution Feedback as Reward

## Context & Reward Intuition

**Why Code Generation is Perfect for RL:**

- Unlike creative measure of conversations: does it pass tests?

**Reward Spectrum:**


No human labeling needed! The Python interpreter tells us if code works.

## Formal Reward System

**Formal Reward Function:**

$$r_{\text{code}}(x, y) = \sum_{i=1}^{n} [\text{test}_i \text{ passes}] + \sum_{j} b_j \cdot [\text{bonus}_j \text{ achieved}]$$

where bonus terms $b_j$ include:

- $b_{\text{style}} = 0.5$ if passes linter
- $b_{\text{doc}} = 0.3$ if has docstring
- $b_{\text{efficiency}} = 0.2$ if under time limit

## Specific Implementation & Process

**Concrete Formula:**

$$r = \sum_{i=1}^{n} [[\text{test}_i \text{ passes}] + 0.5 \cdot [\text{no lint errors}] + 0.3 \cdot [\text{has docstring}] + 0.2 \cdot [\text{efficient execution}]]$$

**Evaluation Process** (Plain English):

1. Model generates code solution
2. Execute code with all test inputs
3. Count number of passing tests
4. Check for bonus conditions:
   - ✓ Clean code style (linter)
   - ✓ Good documentation
   - ✓ Efficient execution
5. Sum all points for total reward

## Walkthrough Example

**Concrete Example:** 'Write function to reverse a string'

**Test Cases:**

- $f$ ("hello") = "olleH" Pass
- $f$ ("") = "" Pass
- $f$ ("a") = "a" Pass

**Generated Solution:**

- Function with docstring
- Uses Python slicing: $s[::-1]$
- Passes linter checks
- $O(n)$ time complexity

**Reward Calculation:**

$r = 3$ (tests passed)
$+ 0.5$ (clean linter)
$+ 0.3$ (has docstring)
$+ 0.2$ (efficient)
$= \mathbf{4.0}$

# Practical RL Training for Code Generation

## Training Algorithm:

**1. Iteration Loop:** Sample $n = 1000$ programming problems

**2. Per-Problem ($p$) Loop**

**(a) Generate Solutions**

- Generate group of $k = 4$ code solutions 
  $S = \{s_1, s_2, s_3, s_4\} \sim \text{model}(p)$

**(b) Evaluate and Score Solution $s_i \in S$**

- Execute code with test cases from $p$
  → base reward: $r_{\text{base}} = \frac{\#\text{ tests passed}}{\#\text{ total tests}} \times 10$
- ✅ +0.5 if no linter errors
- +0.3 if has docstring
  – Syntax/Runtime error: $r_i = -5$
  – Otherwise: $r_i = r_{\text{base}} + \text{bonuses}$
- Collect all rewards: $R = \{r_1, r_2, r_3, r_4\}$

**(c) Calculate Advantages (GRPO-style):**

- Compute baseline $b = \frac{1}{k}\sum_{i=1}^{k} r_i$
- Compute advantages $A_i = r_i - b$ for each solution

**(d) Update Policy:**

- For each solution $s_i$ with $A_i > 0$:
  – Update model: $\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(s_i|p) \cdot A_i$
  (Only train on solutions better than group average)

## Evaluation:

**Evaluation Process**

- Test model on held-out problems
- Report pass@1 accuracy (success rate with single attempt)

| Iteration | pass@1 | pass@10 |
|---|---|---|
| 0 (Base model) | 35% | 58% |
| 1 | 42% | 65% |
| 2 | 51% | 73% |
| 3 | 59% | 79% |
| 5 | 68% | 85% |
| 10 | 75% | 90% |

**Key Insight:** After 10 iterations of RL training, the model more than **doubles** its success rate (35% → 75%)!

# Math: Formal Verifiers as Reward

## **Intuition:** Two Types of Math Verification

### Numerical Verification (Easier)

For arithmetic and numerical problems:

- ✓ Model generates numerical answer
- ✓ Compare to ground truth


**Binary Reward:**
correct = +1, incorrect = 0

### Formal Proof Verification (Harder)

For proofs and formal mathematics:

- ✓ Model generates proof in formal language (Lean, Coq, Isabelle)
- ✓ Proof checker verifies logical validity
- ✓ Reward based on proof correctness


The second type is particularly exciting because it enables models to do ***real mathematics***, not just arithmetic!

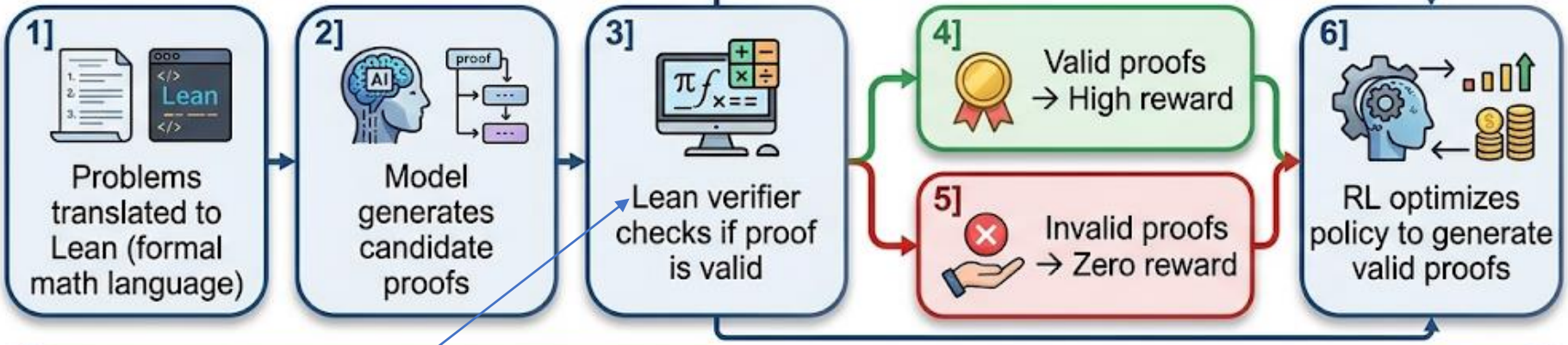
# Formal Mathematics with Lean

## AI System: AlphaProof and IMO 2024

In July 2024, Google DeepMind's AlphaProof (using RL with Lean theorem prover) solved **4** out of **6** problems from the International Mathematical Olympiad, earning a silver medal performance!

IMO 2024

**4** of **6** problems solved




**The breakthrough:**

**Lean** is an open-source interactive theorem prover and functional programming language developed by Microsoft Research in 2013. It is used to digitize and formally verify mathematical proofs and complex software systems.

This was the first time AI reached medal-level performance on IMO, one of the hardest math competitions in the world! The key was first time AI reached medal-educanted as the reount in an times.

The key was using ***formal verification*** as the reward signal—completely objective, no human judgment needed.

# Formal Proof Verification Algorithm

**Formal Verification Reward (r_proof)**

For theorem $\theta$ and proof $\pi$:

$$r_{\text{proof}}(\theta, \pi) = \begin{cases} +10 & \text{if } \text{Verify}(\theta, \pi) = \text{True} \\ 0 & \text{otherwise} \end{cases}$$

**Partial Credit Variant**

For proof with $n$ steps:

$$r_{\text{proof}}(\theta, \pi) = \sum_{i=1}^{n} [\text{step}_i \text{ is valid}]$$

This rewards partially correct proofs, providing a richer learning signal during training.

**Optional Bonus Terms:**

- $+b_{\text{elegance}}$ for concise proofs
- $+b_{\text{generality}}$ for general theorems
- $+b_{\text{novelty}}$ for novel techniques

**Plain English Interpretation**

**Verification Process:**

1. State theorem in formal language
2. Model generates proof (sequence of logical steps)
3. Automated proof checker validates each step:
   - Does this follow from previous steps?
   - Are all inference rules correctly applied?
   - Does the conclusion match the goal?
4. If all steps valid → Proof accepted → High Reward
5. If any step invalid → Proof rejected → No Reward

**Concrete Example**

**Theorem:** For all natural numbers $n$: $n + 0 = n$

**Generated Proof** (in Lean):

- Claim: $n + 0 = n$ for arbitrary $n \in \mathbb{N}$
- Justification: By reflexivity (definition of addition)
- Conclusion: Theorem holds

**Lean Proof Checker:** Valid proof

**Reward Calculation:** $r$ = +10

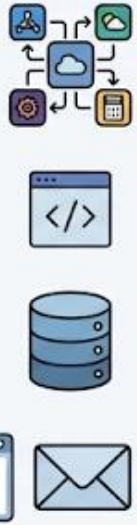
- Verification: Pass → $r = +10$

# Tool Use: API Success as Reward

## Tool-Using AI:

Modern AI systems don't just generate text—they use tools:

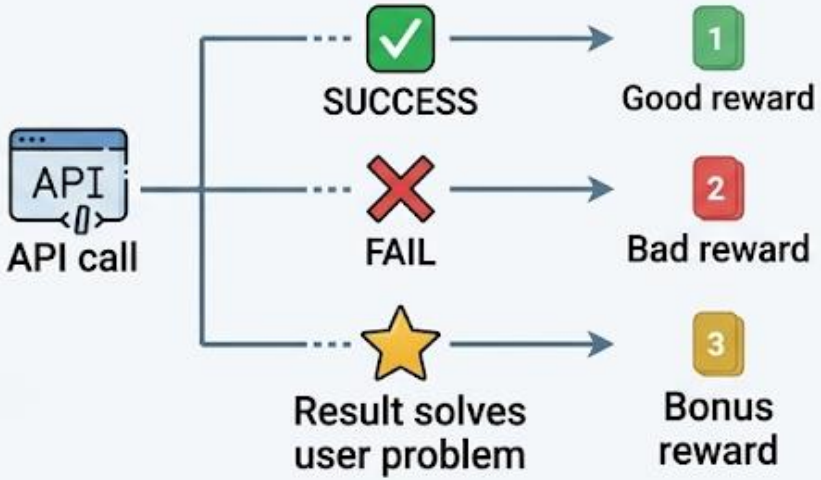
- **Call APIs** (weather, search, calculator)
- **Execute code** (Python interpreter)
- **Query databases** (SQL)
- **Control applications** (browser, email)



## The RL opportunity:

Each tool interaction gives immediate feedback:

- **API call succeeds → Good reward**




This enables models to learn **effective tool use through RL!**

# Breaking It Down Step-by-Step

## Tool Use RL Example:

**User Query:** "What's the weather like in Tokyo right now?"

**Required Task Sequence:**

- ✓ Recognize need to call weather API
- ✓ Format API call with correct parameters
- ✓ Parse API response
- ✓ Present information clearly to user

## Attempt 1 (Novice Model):

1. **Action:** API Call weather API
   **Parameters:** "Tokyo weather now"
2. **Result:** ❌ ERROR – Invalid parameter format
3. **Reward:** $r_1 = -5$ *(API call failed)*
4. **Total Reward:** −5

## Attempt 2 (Learning):

1. **Action:** API Call weather API
   - Parameters: {**city**: "Tokyo", country: "Japan"}
2. **Result:** ✅ Success
   - **Response:** {**temp**: 22, condition: "Sunny", humidity: 65}
   - **Reward:** +5 *(API call succeeded)*
3. **Generated Response:** 💬
   - "The temperature in Tokyo is 22°C with sunny skies."
   - **Reward:** +3 *(basic formatting)*
4. **Total Reward:** $r_2 = +8$

## Attempt 3 (Expert Model):

1. **Action:** API Call weather API
   - Parameters: {**city**: "Tokyo", country: "JP", untis: "metric"}
2. **Result:** ✅ Success with comprehensive data
   - **Response:** {**temp**: 22, condition: "Sunny", humidity: 65, **wind_speed**: 3.5, **current_time**: "2026-03-29 23:05:46 CDT"}

💬 **Generated Response:** 💬

- "As of 2026-03-29 at 23:05:46 CDT, the current weather in Tokyo is 22°C and sunny, with 65% humidity and light winds at 3.5 m/s."
- **Reward:** +5 *(comprehensive and well-formatted)*

4. **Total Reward:** $r_3 = +10$

## Learning Outcomes

- Format API parameters correctly (Structured Data)
- Select appropriate tools for queries
- Present results clearly and comprehensively
- Handle API errors gracefully

# Multi-Tool Orchestration

**Complex Tool Use: Chaining Multiple Tools**
Real tasks often require multiple tools in sequence:

**Example**: 'Book me a flight to Paris next month'
Required tools:

1. 📅 Calendar API (check availability next month)
2. ✈️ Flight search API (find flights)
3. 🛄 Booking API (reserve flight)
4. ✉️ Email API (send confirmation)

**Penalties:**

- Wrong API parameters: -2
- Skip necessary step: -5
- Book wrong dates: -10
- Exceed budget: -15

**RL reward structure:**

| Milestone | Reward |
|---|---|
| Check calendar successfully | +1 |
| Identify available dates | +2 |
| Search flights for right dates | +2 |
| Find flights in budget | +2 |
| Successfully book | +5 |
| Send confirmation | +1 |
| Complete task correctly | +7 |
| **Maximum total** | **+20** |

**Model learns to:**

- Plan multi-step actions
- Check preconditions
- Handle dependencies
- Recover from errors

# LLMs Can Teach Themselves to Use Tools

- We can use SFT to train LLMs to utilize the tools.

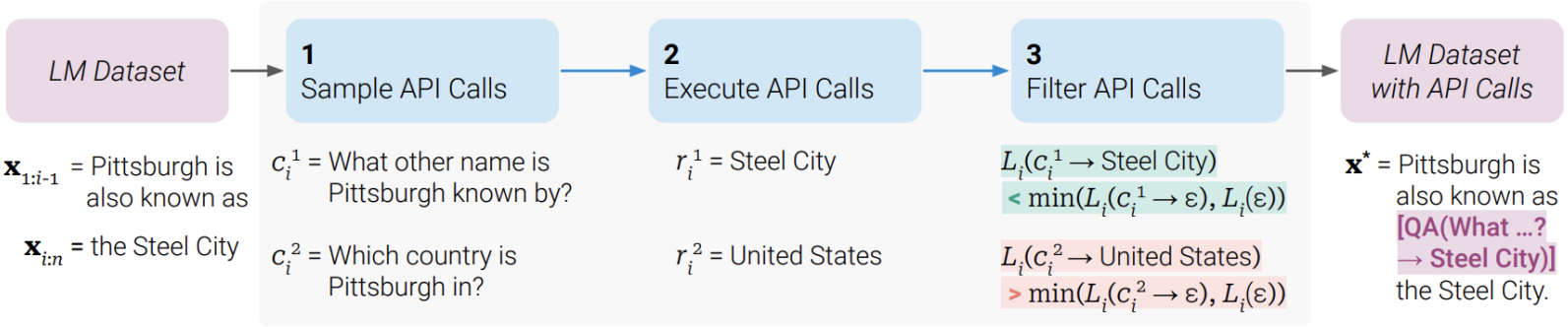



Figure 2: Key steps in our approach, illustrated for a *question answering* tool: Given an input text $\mathbf{x}$, we first sample a position $i$ and corresponding API call candidates $c_i^1, c_i^2, \ldots, c_i^k$. We then execute these API calls and filter out all calls which do not reduce the loss $L_i$ over the next tokens. All remaining API calls are interleaved with the original text, resulting in a new text $\mathbf{x}^*$.

# Multi-Turn Dialogue: Conversation Quality

Most RLHF focuses on single-turn interactions: user asks, model responds, done.
But real conversations have multiple turns! How do we apply RL to improve multi-turn dialogue quality?

- Reward must consider entire conversation, not just one response
- Early responses affect later ones (long-term consequences)
- Credit assignment problem: which response was good/bad?

**The Solution:**

- Assign rewards at conversation level
- Use techniques like TD-learning to propagate credit
- Consider factors like:
  - Information gathering progress
  - Task completion
  - User satisfaction signals

Imagine a conversation as a sequence of states. Each time the model gives a response, it makes a "guess" at how successful this conversation will eventually be (the **Value Estimate**).

- **Turn 1:** The model asks a clarifying question. It thinks: *"I'm about 60% likely to solve this."*
- **Turn 2:** The user provides the missing info. The model now thinks: *"Great, I'm now 90% likely to solve this!"*
- **The TD Update:** The model looks at the **difference** between its 90% guess and its 60% guess. That $+30\%$ difference is a "TD Error." It immediately "propagates" that success back to Turn 1, rewarding the clarifying question because it led to a better state.

# Multi-Turn Dialogue Algorithm ⚙️

**Formal Multi-Turn Reward:**

For conversation $C = \{(u_1, r_1), \ldots, (u_t, r_t)\}$:

**Terminal reward:**

$$R_{\text{term}}(C) = \begin{cases} +10 & \text{task completed} \\ +5 & \text{partial progress} \\ 0 & \text{no progress} \\ -5 & \text{task failed} \end{cases}$$

**Per-turn rewards:**

$$r_t = r_{\text{quality}}(r_t) + r_{\text{progress}}(C_{1:t})$$

**Total reward:**

$$R_{\text{total}}(C) = \sum_{t=1}^{T} \gamma^{T-t} r_t + R_{\text{term}}(C)$$

where $\gamma$ is discount factor (typically 0.95-0.99).

Earlier turns get discounted more (focus on recent quality).

**Plain English:**

**Conversation reward has two parts:**

1. **Per-turn rewards:**
   - Quality of each response
   - Progress toward goal 🎯
   - User engagement

   Each turn gets a small reward based on local quality.
2. **Terminal reward:**
   - Did conversation achieve goal?
   - Large reward at end if successful

**Example: Tech support conversation**

**Turn 1:** "What's the problem?"
→ **Local reward:** +1 (good question)
**Turn 2:** "Have you tried restarting?"
→ **Local reward:** +2 (standard diagnostic)
**Turn 3:** "That fixed it!"
→ **Terminal reward:** +10 (problem solved!)
**Total: +13** 👍

**Model learns:** asking good diagnostic questions early leads to successful resolution!

# Breaking It Down Step-by-Step

**Scenario:** User needs help with password reset

## Conversation 1 (Poor)

| Turn | Exchange | Reward |
|---|---|---|
| 1 | User: "I can't log in"<br>Bot: "Have you tried using Incognito mode?" | —<br>+1 |
| 2 | User: "That doesn't help"<br>Bot: "Clear your cookies" | —<br>+1 |
| 3 | User: "Still can't log in!"<br>Bot: "Try a different browser" | —<br>0 |
| 4 | User: "This isn't working. Bye."; (Task failed) | −5 |
| **Total reward:** | | **-3** |

Bot never diagnosed the actual problem (forgotten password)!

## Conversation 2 (Good)

| Turn | Exchange | Reward |
|---|---|---|
| 1 | User: "I can't log in"<br>Bot: "Are you getting an error message?" | —<br>+2 |
| 2 | User: "It says 'incorrect password'"<br>Bot: "Let me help reset your password..." | +3 |
| 3 | User: "Got the reset email , thanks!"<br>Bot: "Great! Let me know if you need anything else." (Task completed) | +10 |
| **Total reward:** | | **+17** |

Bot quickly diagnosed and solved the problem!

## What the model learns:

- Diagnostic questions early → higher terminal reward
- Random suggestions without diagnosis → low reward
- Quick resolution → bonus reward
- User satisfaction signals ("thanks!") → positive signal